

Configuration d'un IDE VS Code pour C/C++

- 1. Création d'un Hello World simple
- 2. Gestion d'un projet complet
 - 2.1. Ajout de fichiers sources
 - 2.2. Ajout d'une configuration d'exécution
 - 2.3. Recompiler automatiquement avant exécution
 - 2.4. Mode debug et release
- 3. Annexes
 - 3.1. Documentation utile

1. Création d'un Hello World simple

Rien de tel que de se référer à la documentation officielle pour commencer:

<https://code.visualstudio.com/docs/languages/cpp>.

Quelques recommandations complémentaires à cette documentation :

- Installation d'un compilateur C/C++ :
 - Sous Windows :
 - Pour enregistrer le chemin du compilateur dans la variable d'environnement **PATH** :
 - TODO
 - Penser à relancer VS Code après avoir modifié la variable d'environnement **PATH**.

En effet, les variables d'environnement d'un nouveau processus ne sont prises en compte qu'au lancement du processus, et reprennent l'environnement de leur processus père.

En l'occurrence, il convient de relancer VS Code pour que celui ci prenne en compte les variables d'environnement de Windows fraîchement mises à jour.

- Sous Linux :
 - Rien de plus à faire pour **gcc**.

L'architecture du système de fichier installe généralement les nouveaux binaires dans un répertoire déjà enregistré dans la variable d'environnement **PATH**.

Exemple sur Ubuntu :

```
$ cat /etc/issue
Ubuntu 24.04.3 LTS \n \l
```

```
$ sudo apt install gcc
...

$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying
conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A
PARTICULAR PURPOSE.

$ which gcc
/usr/bin/gcc

$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
:/usr/games:/usr/local/games:/snap/bin:/snap/bin:/mnt/Data/T
ools/file-utils/bin
```

- Pour `g++` toutefois, il semble qu'il faille redémarrer VS code, pour la prise en compte des `include paths` C++.

Après obtention d'un programme `helloworld` qui compile et s'exécute sans erreur, noter le fichier `.vscode/tasks.json` automatiquement créé:

```
{
  "tasks": [
    {
      "type": "cppbuild",
      "label": "C/C++: g++ build active file",
      "command": "/usr/bin/g++",
      "args": [
        "-fdiagnostics-color=always",
        "-g",
        "${file}",
        "-o",
        "${fileDirname}/${fileBasenameNoExtension}"
      ],
      "options": {
        "cwd": "${fileDirname}"
      },
      "problemMatcher": [
        "$gcc"
      ],
      "group": {
        "kind": "build",
        "isDefault": true
      },
      "detail": "Task generated by Debugger."
    }
  ]
}
```

```
],  
  "version": "2.0.0"  
}
```

Noter l'utilisation de l'option `-g` qui active le debugging avec `gcc / g++`.

2. Gestion d'un projet complet

L'exemple proposé reste un peu simpliste, dans la mesure où on a :

- un seul fichier source,
- une règle de build qui compile le fichier courant.

La structure usuelle d'un projet est plutôt :

- plusieurs fichiers source,
- une arborescence de répertoires permettant de séparer les différents types de fichiers,
- une règle de build qui compile le programme décrit par l'ensemble des sources du projet en mode release.
- une règle de build qui compile le programme également, en mode debug.

2.1. Ajout de fichiers sources

Ajouter les répertoires suivants dans le projet :

- `build/`
- `doc/`
- `src/`

Déplacer le fichier `helloworld.cpp` dans le répertoire `src/`, et supprimer le binaire généré précédemment.

Ajouter les fichiers suivants dans le répertoire `src/` :

```
// whoami.h  
  
#ifndef _WHOAMI_H_  
#define _WHOAMI_H_  
  
const char* const whoami();  
  
#endif
```

```
// whoami.cpp  
  
#include "whoami.h"
```

```
const char* const whoami() {
    return "you";
}
```

Modifier le fichier `helloworld.cpp` comme suit:

```
// helloworld.cpp

#include <iostream>
#include "whoami.h"

int main()
{
    std::cout << "Hello " << whoami() << "!" << std::endl;
}
```

Modifier le fichier `.vscode/tasks.json`:

- Modifier le *label* de la tâche de `C/C++: g++ build active file` en `C/C++: g++ build helloworld (debug)` pour indiquer dans le nom qu'on compile désormais le programme principal du projet, et non plus le fichier courant.
- Modifier la liste des fichiers source de `"${file}"` à `"src/*.cpp"`.
- Modifier la destination du binaire de sortie (après l'option `-o`) de `"${fileDirname}/${fileBasenameNoExtension}"` en `"build/helloworld.debug.exe"` pour générer dans le répertoire `build/`.
- Modifier la ligne *detail* de `Task generated by Debugger.` en `Compile in Debug mode.` pour expliciter l'objet de la tâche de build.

2.2. Ajout d'une configuration d'exécution

Sélectionner un fichier `.cpp`, et appuyer sur F5 :

- Notre tâche ne permet plus d'exécution implicite.
- Sélectionner l'option "Open 'launch.json'" pour pouvoir créer une configuration d'exécution explicite.
 - Un fichier `.vscode/launch.json` est créé.
- Run > Add Configurations...
- Sélectionner l'option "C/C++: (gdb) Launch".
- Ajuster les paramètres comme suit :
 - `program` : `${workspaceFolder}/build/helloworld.debug.exe` pour expliciter le programme à exécuter.
 - `cwd` : `${workspaceFolder}/build` pour utiliser le répertoire `build/` comme répertoire courant d'exécution.

Appuyer sur F5, à partir de n'importe quel fichier : VS Code lance le programme et affiche "Hello you!".

2.3. Recompiler automatiquement avant exécution

Modifier Le fichier `whoami.cpp` pour modifier le texte renvoyé par la fonction `whoami()`, puis appuyer sur F5 :

- Le programme affiche toujours "Hello you!".

Dans le fichier `.vscode/launch.json`, ajouter l'option `"preLaunchTask": "C/C++: g++ build helloworld (debug)"` dans notre configuration.

Appuyer sur F5 :

- La compilation du programme est lancée.
- L'exécution du programme prend en compte la modification du fichier `whoami.cpp`.

2.4. Mode debug et release

Les étapes précédentes nous ont amené à configurer la compilation et l'exécution en mode debug du programme.

TODO :

- Ajouter une tâche dans le fichier `.vscode/task.json` pour compilation en release, i.e. sans l'option `-g`.
- Ajouter une configuration dans le fichier `.vscode/launch.json` pour lancement automatique de la tâche précédente, et exécution du binaire release généré.
 - Faire en sorte que cette dernière soit utilisée lorsqu'on appuie sur CTRL+F5.

3. Annexes

3.1. Documentation utile

Documentation officielle VS Code :

- <https://code.visualstudio.com/docs/languages/cpp>